МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
“Национальный исследовательский университет ИТМО”

**ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ**

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине
“Бизнес-логика программных систем”

Вариант: 3.

Работу выполнили:
Студенты потока 1.5
Коломиец Никита Сергеевич
Болорболд Аригуун
Лектор:
Цопа Евгений Алексеевич
Практик:
Кривоносов Егор Дмитриевич

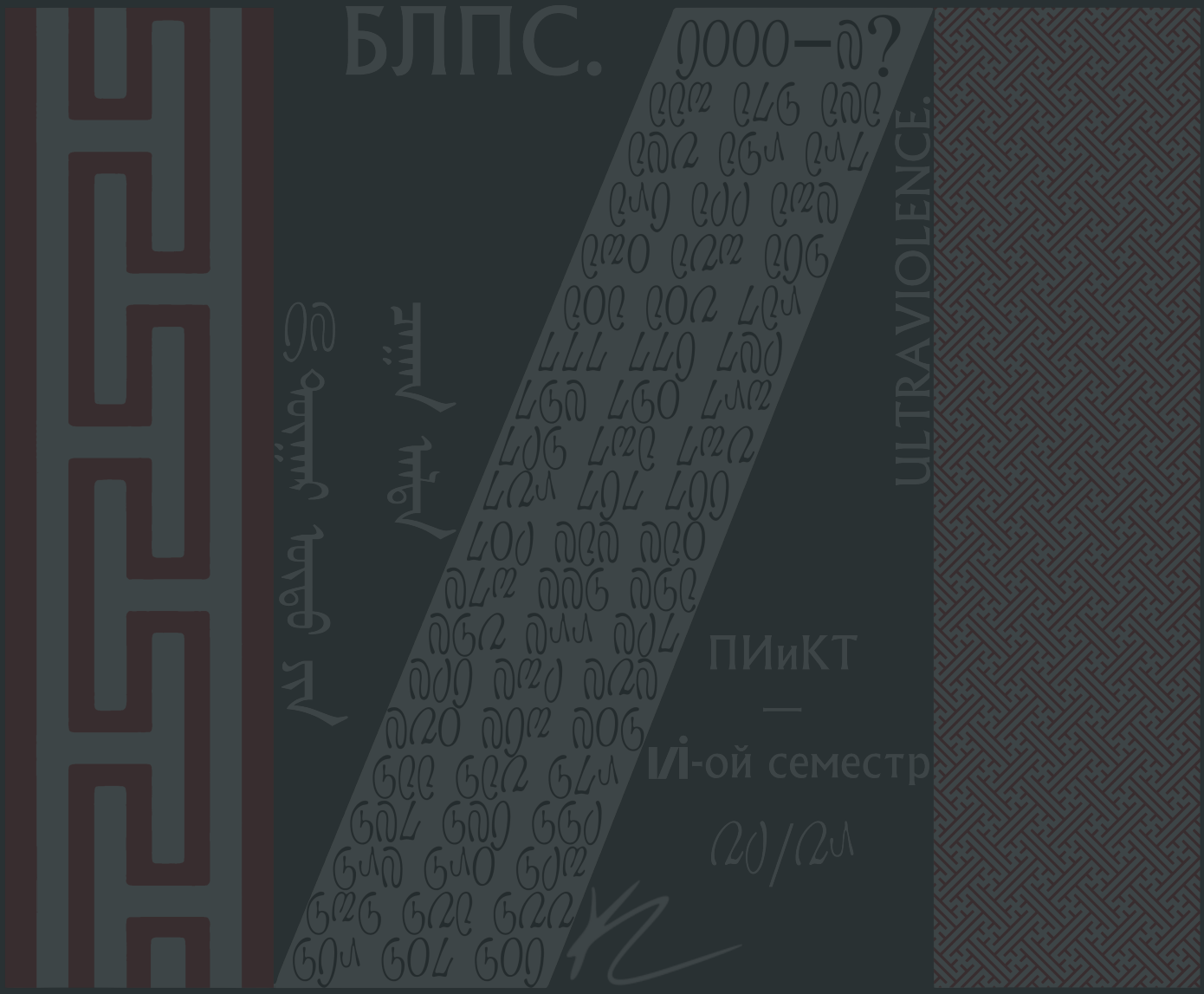
г. Санкт-Петербург
2025 г.

Содержимое

1 Текст задания **3**
1.1 Требования к реализации аснихронной обработки: 3
1.2 Требования к реализации распределенной обработки: 3
1.3 Требования к реализации периодических задач по расписанию: 3
1.4 Требования к интеграции с внешней Корпоративной Информационной Системой (EIS): 3

2 Выполнение **4**
2.1 BPMN-модель 4
2.2 UML 4
2.3 Периодические задачи по расписанию 4
2.4 Интеграция с EIS 4

3 Вывод **5**



1 Текст задания

Доработать приложение из лабораторной работы #2, реализовав в нём асинхронное выполнение задач с распределение бизнес-логики между несколькими вычислительными узлами и выполнением периодических операций с использованием планировщика задач, а также интеграцию с внешней информационной системой.

1.1 Требования к реализации аснихронной обработки:

1. Перед выполнением работы необходимо согласовать с преподавателем набор прецедентов, в реализации которых целесообразно использование распределенного выполнения задач. Если таких прецедентов использования в имеющейся бизнес-процесса нет, нужно согласовывать реализацию новых прецедентов, доработав таким образом модель бизнес-процесса из лабораторной работы #1.
2. Асинхронное выполнение задач должно использовать модель доставки "очередь сообщений".
3. В качестве провайдера сервиса асинхронного обмена сообщениями необходимо использовать очередь сообщений на базе `RabbitMQ`.
4. Для отправки сообщений необходимо использовать протокол `MQTT`. Библиотеку для реализации отправкии сообщений можно взять любую на выбор студента.
5. Для получения сообщений необходимо использовать `JMS API`.

1.2 Требования к реализации распределенной обработки:

1. Обработка сообщений должна осуществляться на двух независимых друг от друга узлах сервера приложений.
2. Если логика сценария распределенной обработки предполагает транзакционность выполняемых операций, они должны быть вылючены в состав распределенной транзакции.

1.3 Требования к реализации периодических задач по расписанию:

1. Согласовать с преподавателем прецедент или прецеденты, в рамках которых выглядит целесообразным использовать планировщик задач. Если такие прецеденты отсутствуют — согласовать с преподавателем новые и добавить их в модель автоматизуемого бизнес-процесса.
2. Реализовать утверждённые прецеденты с использованием планировщика задач Spring (`@Scheduled`).

1.4 Требования к интеграции с внешней Корпоративной Информационной Системой (EIS):

1. Корпоративная Информационная Система, с которой производится интеграция, а также её функциональные возможности выбираются на усмотрение преподавателя и согласуются с ним.
2. Взаимодействие с внешней Копроративной Информационной Системой должно быть реализовано с помощью технологии `JCA` (Jakarta Connectors).

Правила выполнения работы:

1. Все изменения, внесённые в реализуемый бизнес-процесс, должны быть учтены в описывающей его модели, REST API и наборе скриптов для тестирования публичных интерфейсов модуля.
2. Доработанное приложение необходимо развернуть на сервере `helios`, либо продемонстрировать его работоспособность на собственной инфраструктуре.

2 Выполнение

Сслыка на репозиторию: GitHub
XVIIStarPlatinum — Аригуун;
nkolomiika — Никита;

2.1 BPMN-модель

Можно посмотреть в этом ссылке.

2.2 UML

Ссылка на первый сервис
Ссылка на второй сервис (почтовый)

2.3 Периодические задачи по расписанию

В бизнес-процессе этого проложения рассматриваются 2 прецедентов для периодических задач:

1. Ежемесячное обнуление месячного бюджета, в результате которого пользователь снова может потратить деньги;
2. Ежедневное удаление пользовательских подписок из базы данных.

```
@Scheduled(cron = "0 0 0 1 * *")
public void resetAllSpending() {
    userBudgetRepository.findAll().forEach(user -> {
        user.setCurrentSpending(0D);
        user.setLastLimitReset(LocalDate.now());
        userBudgetRepository.save(user);
    });
}

@Scheduled(cron = "0 0 0 * * ?")
public void processCancelledSubscriptions() {
    userSubscriptionRepository.deleteUserSubscriptionsPeriodically();
}
```

Также рассматривается сам механизм снятия средств из карты в случае с подпиской. Срок снятия зависит от срока, установленного в самом подписке.

2.4 Интеграция с EIS

Битрикс24 — единая цифровая платформа («командный центр») для управления бизнесом, объединяющая в одном окне CRM, мессенджер, задачи, конструктор сайтов и аналитику. В этом работе оно используется для управления пользовательскими картами:

```
String jsonBody = "{ \"fields\": { " +
        "\"ENTITY_ID\": 2, " +
        "\"NAME\": \"" + card.getUserId() + "\", " +
        "\"XML_ID\": \"" + card.getId() + "\", " +
        "\"RQ_COR_ACC_NUM\": \"" + card.getExpiryDate().getDayOfMonth() + "/" + card.getExpir
        "\"RQ_ACC_NUM\": \"" + card.getNumber() + "\" " +
        "} }";
```

```
BitrixConnection connection = null;
try {
    connection = bitrixConnectionFactory.getConnection(token);
    connection.callApi(endpoint, jsonBody);
```

```
    } catch (Exception e) {
        throw new RuntimeException("Ошибка при добавлении карты в Bitrix24", e);
    } finally {
        if (connection != null) {
            connection.close();
        }
    }
```

3 Вывод

В процессе выполнения лабораторной работы мы продолжили процесс обучения через реализацию новых аспектов в бизнес-процессах, а именно:

- Асинхронное выполнение задач через брокера сообщений;
- Периодические задачи (cronjobs);
- Интеграция с внешней корпоративной ИС.

Периодические задачи уже были выполнены при первом ЛР, поэтому пришлось реализовать только оставшиеся части. Для выполнения ЛР не требовалось разделить существующую ИС симметричным образом, достаточно было вынести маленькую сервис про уведомлении.